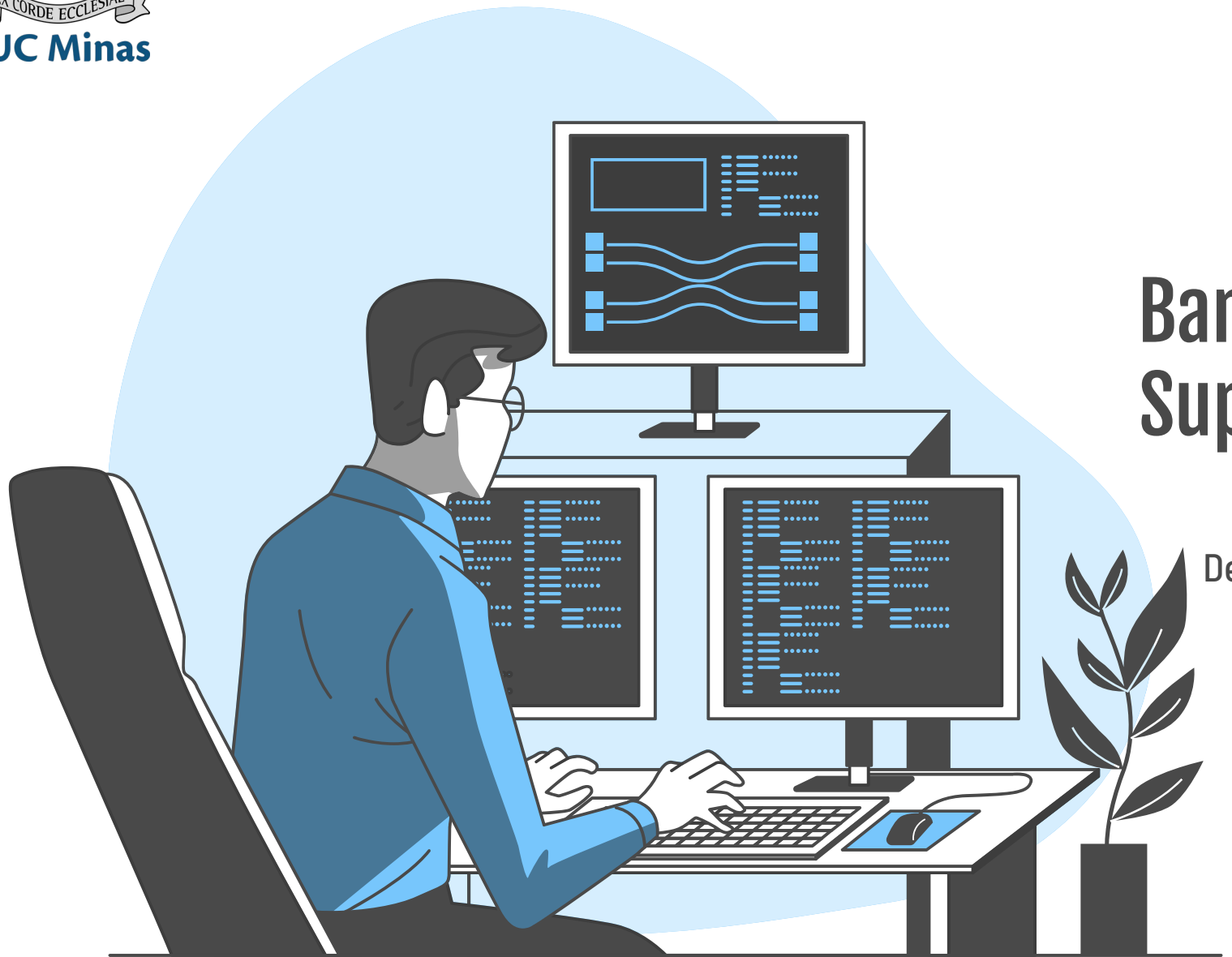
PUC Minas



Aula 06 · Back-end

Banco relacional na nuvem: Supabase + Spring Data JPA

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Entender o que é o **Supabase** e quando usar um banco relacional
2. Criar um projeto no Supabase e obter a *connection string* **JDBC**
3. Configurar `spring.datasource.*` e o **Hibernate** no `application.properties`
4. Mapear entidades com `@Entity` e criar um `JpaRepository`
5. Usar o **Table Editor** e o **SQL Editor** do Supabase

Antes de começar

- Aula 05: API REST
- Front-end, aula 09 (primeiro contato com o Supabase)
- Conta no Supabase (login com GitHub)

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Relacional x não relacional

02 O que é o Supabase

03 Passo 1: criar (ou reutilizar) o projeto

04 Passo 2: copiar a connection string

05 Passo 3: configurar o projeto Spring

06 Passo 4: entidade e repository

07 Passo 5: conferir no painel

08 Aprofundando: relacionamentos e desempenho com JPA

01

Relacional x não relacional

Relacional x não relacional

Nos projetos usamos dois tipos de banco. Escolha pelo formato dos dados:

	Supabase (PostgreSQL)	MongoDB Atlas
Modelo	Tabelas com colunas fixas, linhas	Coleções de documentos JSON flexíveis
Relacionamentos	Chaves estrangeiras e `JOIN`	Documentos aninhados ou referências
Linguagem	SQL	Consultas em JSON / métodos do driver
No Spring	`spring-boot-starter-data-jpa` + `JpaRepository`	`spring-boot-starter-data-mongodb` + `MongoRepository`
Bom para	Dados estruturados, integridade, relatórios	Dados que variam, protótipos rápidos, logs

02

O que é o Supabase

O que é o Supabase

O **Supabase** é uma plataforma que entrega um **PostgreSQL completo na nuvem**, com painel web, editor de tabelas, editor SQL, autenticação, armazenamento de arquivos e uma API REST gerada automaticamente. O plano gratuito permite 2 projetos ativos. Na aula 09 do Front-end você usou o Supabase **pelo navegador**, através da Data API e da chave publicável. Agora o uso é outro: para o back-end Spring Boot, o Supabase é simplesmente **um banco PostgreSQL**, e o Spring conecta direto nele via JDBC, com usuário e senha do banco.

02

O que é o Supabase

Aula 06 · Back-end

O que é o Supabase

Atenção

No plano gratuito, projetos sem uso por **7 dias** são pausados. Basta entrar no painel e clicar em **Restore project**. Lembre disso antes de apresentar um trabalho!

03

Passo 1: criar (ou reutilizar) o projeto

Passo 1: criar (ou reutilizar) o projeto

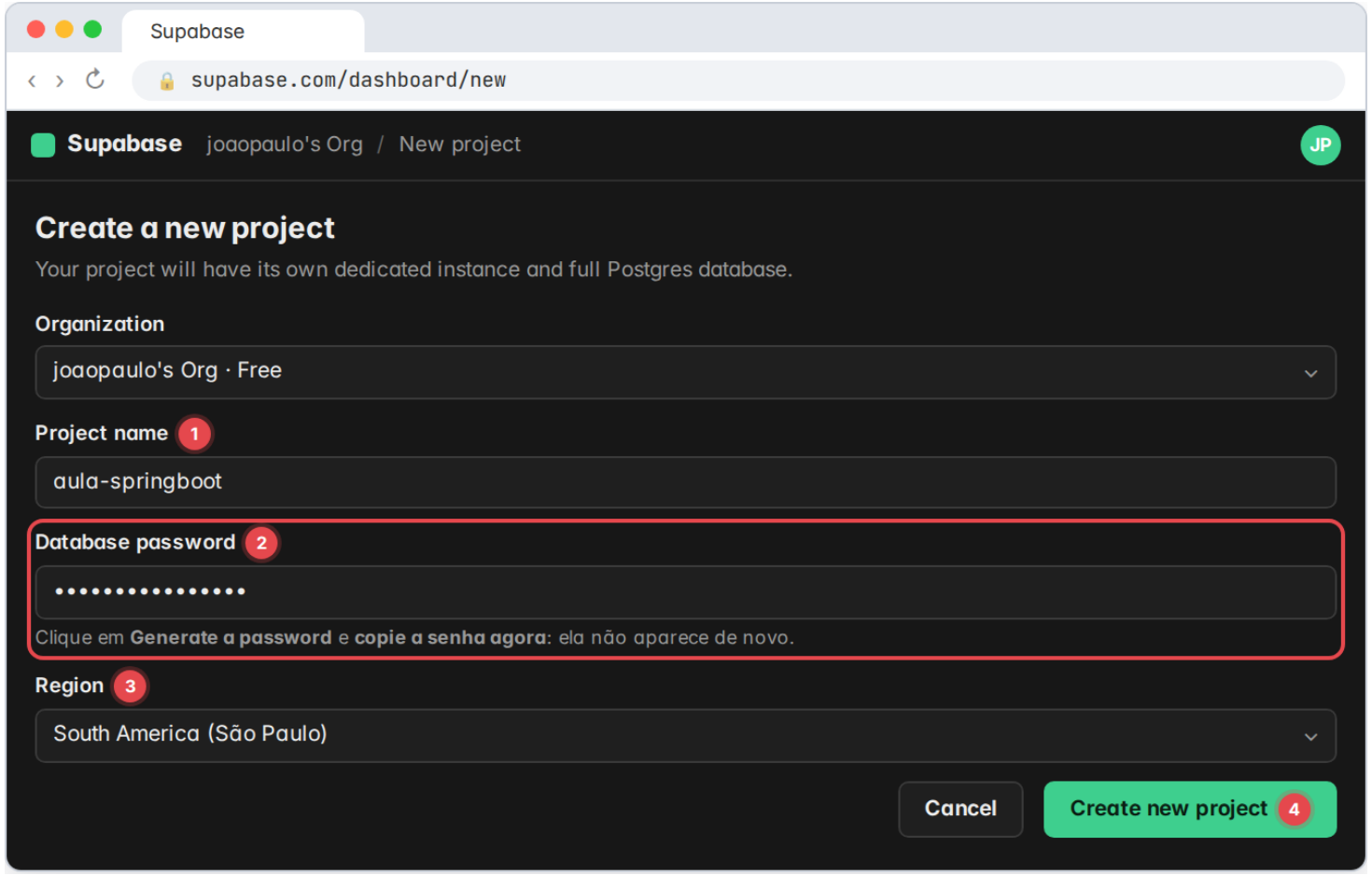
Você já criou um projeto Supabase na aula 09 do Front-end. Pode **reutilizá-lo**: a tabela `livro` desta aula convive com a tabela `tarefas` no mesmo banco. Só vai precisar da **senha do banco** que guardou naquela aula (se esqueceu, redefina em *Database* → *Settings*). Se preferir um projeto separado, o passo a passo abaixo serve de revisão.

03

Passo 1: criar (ou reutilizar) o projeto

Aula 06 · Back-end

Passo 1: criar (ou reutilizar) o projeto



Criação de um projeto no Supabase.

Passo 1: criar (ou reutilizar) o projeto

1. Acesse **supabase.com**, entre com o GitHub e clique em **New project**. Dê um nome ao projeto.
2. Gere a **senha do banco** e guarde em um lugar seguro. Ela será o `DB_PASSWORD`.
3. Escolha a região **São Paulo** (menor latência para o Brasil).
4. Clique em **Create new project** e aguarde 1 a 2 minutos enquanto o banco é provisionado.

04

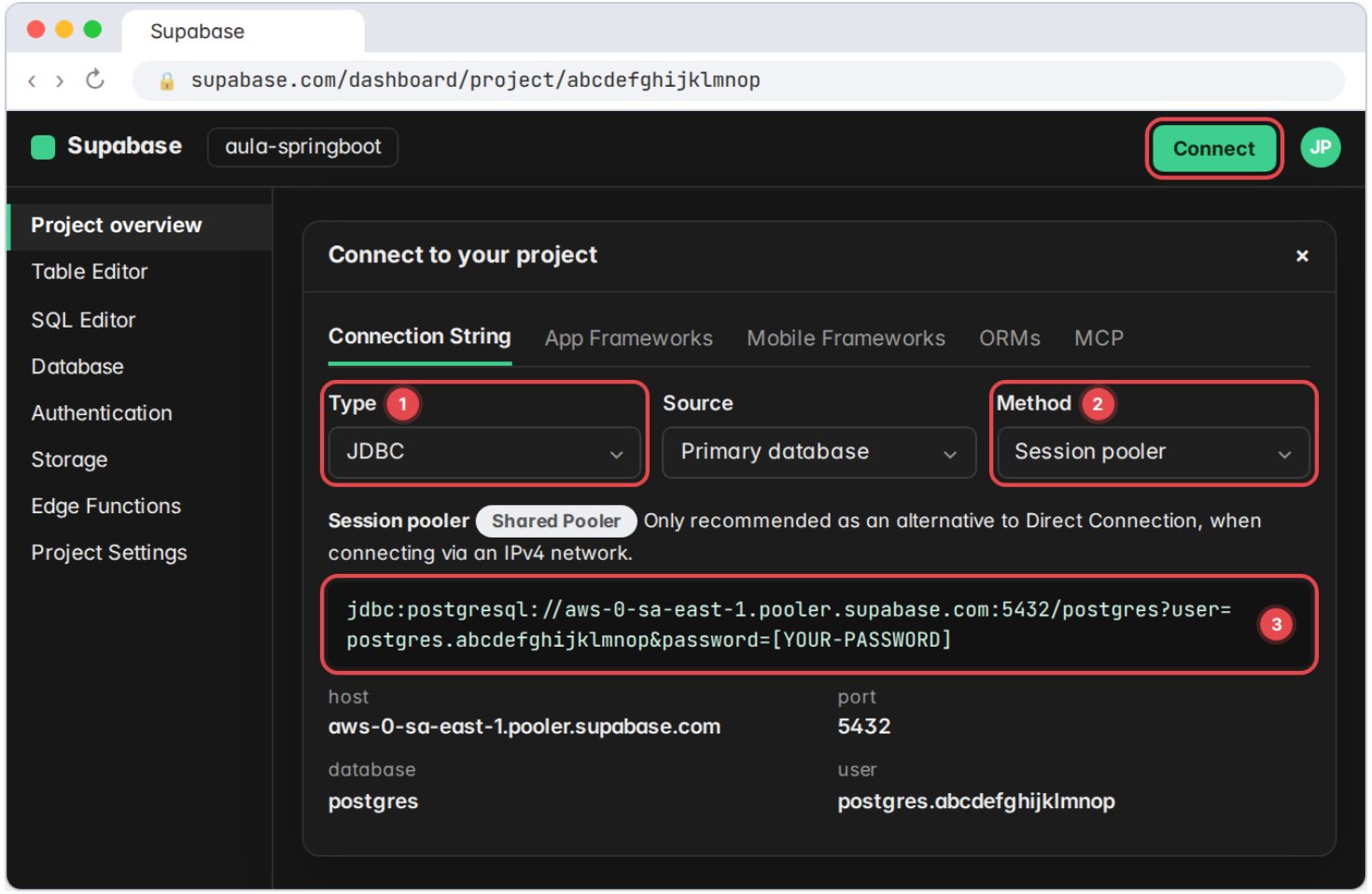
Passo 2: copiar a connection string

Passo 2: copiar a connection string

No topo do painel do projeto clique no botão **Connect**. O Supabase mostra três formas de conexão:

Método	Porta	Quando usar
Direct connection	5432	Só funciona em redes com **IPv6**. Evite: muitas redes domésticas e o Render usam IPv4.
Transaction pooler	6543	Funções *serverless* com muitas conexões curtas
Session pooler	5432	**Use este.** Funciona em IPv4 e com aplicações que ficam ligadas, como o Spring Boot.

Passo 2: copiar a connection string



Janela **Connect**: tipo **JDBC** e método **Session pooler**.

Passo 2: copiar a connection string

1. Em **Type**, escolha **JDBC** (o formato que o Java entende).
2. Em **Method**, escolha **Session pooler**.
3. Copie a URL. Separe a parte antes do `?` (URL), o `user=` (usuário, no formato `postgres.`) e use a senha do passo 1.

Passo 2: copiar a connection string

Erro comum

`UnknownHostException` ou `Connection timed out` ao subir o Spring: você copiou a *Direct connection* (`db.xxxx.supabase.co`), que exige IPv6. Troque para o **Session pooler**. `password authentication failed`: senha errada ou usuário sem o sufixo `.ref-do-projeto`. Se esqueceu a senha, redefina em **Database → Settings → Reset database password**.

05

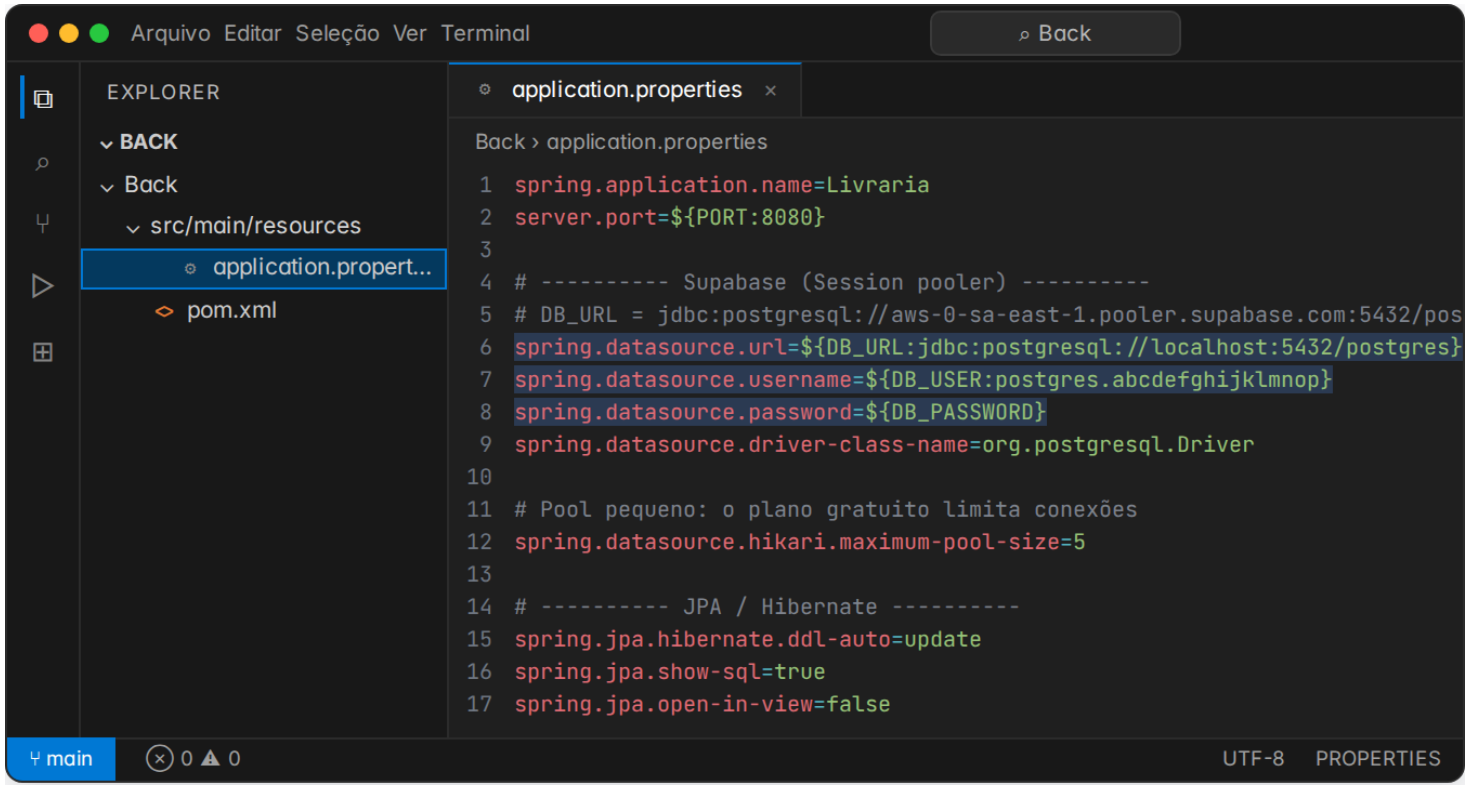
Passo 3: configurar o projeto Spring

Passo 3: configurar o projeto Spring

pom.xml

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

Passo 3: configurar o projeto Spring



Conexão com o Supabase. A URL do *Session pooler* vai na variável `DB_URL` e a senha só em `DB_PASSWORD`.

Passo 3: configurar o projeto Spring

Para rodar localmente com a senha fora do arquivo, defina a variável no terminal antes do comando:

Rodando com a senha em variável de ambiente

```
# Linux / macOS
DB_PASSWORD='sua-senha' ./mvnw spring-boot:run

# Windows PowerShell
$env:DB_PASSWORD="sua-senha"; .\mvnw spring-boot:run
```

Passo 3: configurar o projeto Spring

Dica

No VS Code, você também pode criar `.vscode/launch.json` com `"env": { "DB_PASSWORD": "..." }` para o botão **Run** do Spring Boot Dashboard. Só lembre de manter `.vscode/` no `.gitignore`.

06

Passo 4: entidade e repository

Passo 4: entidade e repository

model/Livro.java

```java
@Entity
@Table(name = "livro")
public class Livro {
    @Id
    @GeneratedValue(strategy =
        GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 120)
    private String titulo;

    private String autor;
    private Integer ano;

    protected Livro() { }   // exigido pelo JPA
    // getters e setters
}
```

repository/LivroRepository.java

```java
public interface LivroRepository
        extends JpaRepository<Livro, Long> {

    List<Livro> findByAutorContainingIgnoreCase(
        String autor);

    List<Livro> findByTituloStartingWith(
        String prefixo);
}
```

Console ao subir (ddl-auto=update)

```
Hibernate: create table livro (
  id bigint generated by default as identity,
  ano integer, autor varchar(255),
  titulo varchar(120) not null,
  primary key (id))
```

Passo 4: entidade e repository

Anotação JPA	Significado
`@Entity`	A classe é mapeada para uma tabela
`@Table(name=...)`	Nome da tabela (padrão: nome da classe)
`@Id` + `@GeneratedValue(IDENTITY)`	Chave primária autoincremento
`@Column`	Restrições da coluna: `nullable`, `unique`, `length`
`@ManyToOne`, `@OneToMany`	Relacionamentos entre tabelas
`@Enumerated(EnumType.STRING)`	Grava o nome do enum, não o número

ddl-auto: quem cria as tabelas?

Valor	Efeito
`update`	Cria tabelas e colunas que faltam. Bom para aulas e protótipos.
`create-drop`	Apaga e recria a cada execução (perde os dados)
`validate`	Só confere se as tabelas batem com as entidades. Use em produção.
`none`	Não mexe em nada. As tabelas são criadas por script SQL.

07

Passo 5: conferir no painel

Passo 5: conferir no painel

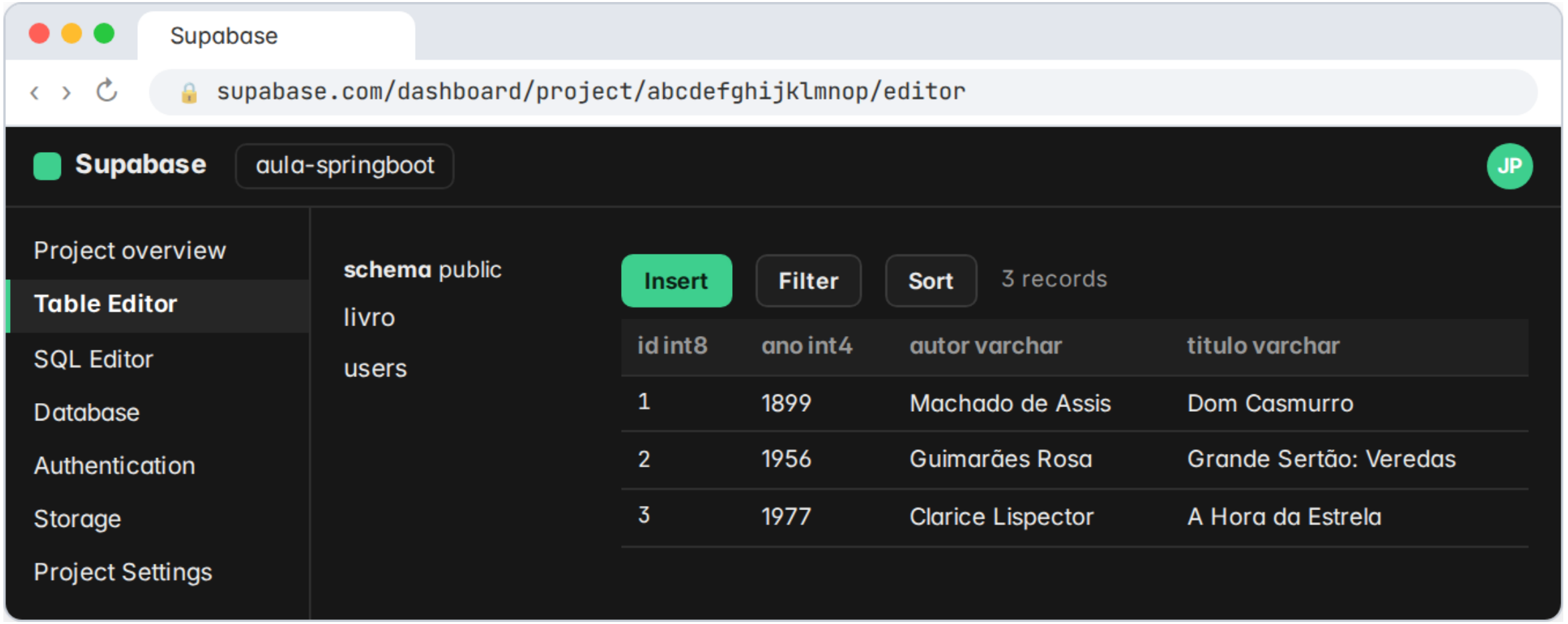
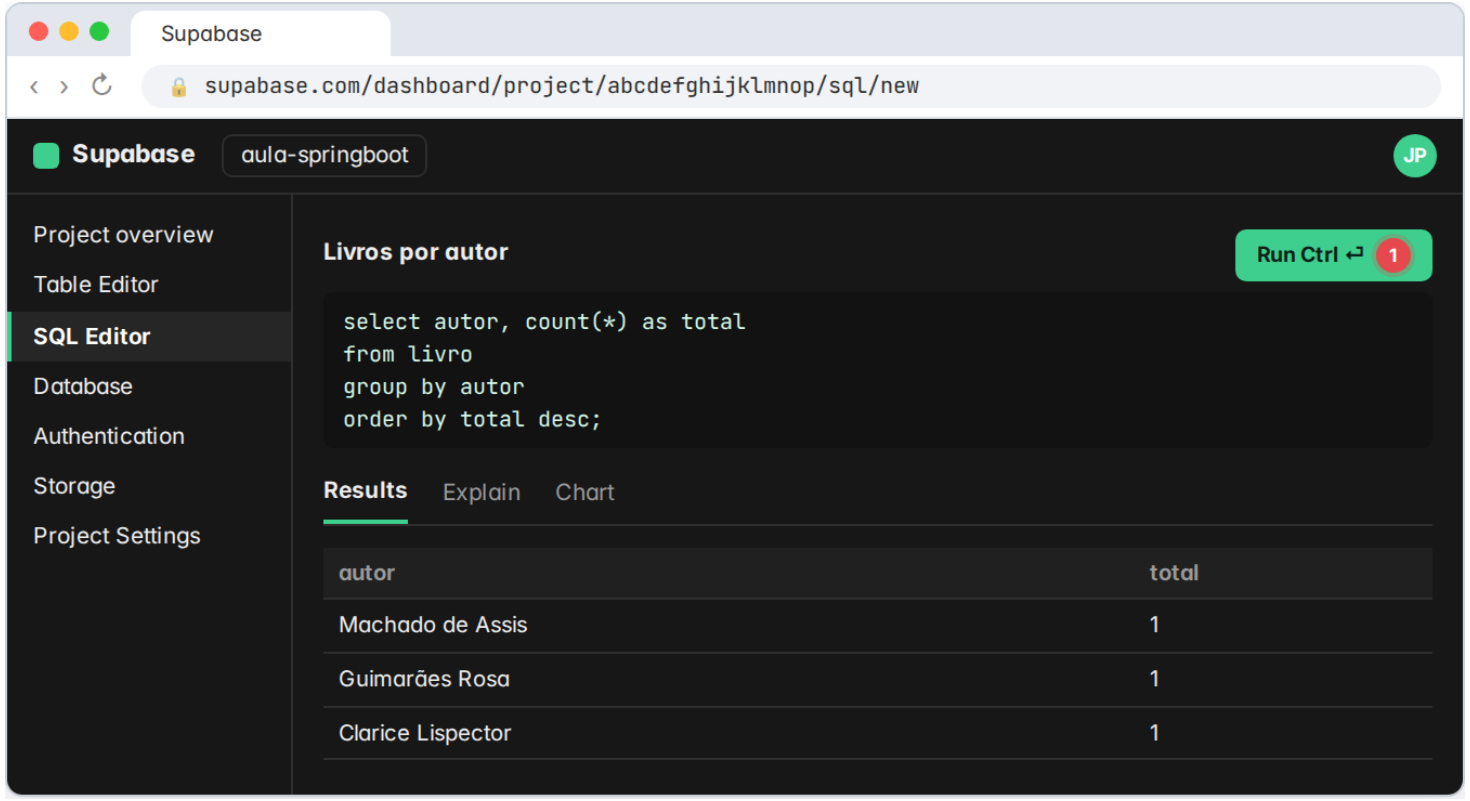


Table Editor: a tabela `livro` criada pelo Hibernate, com registros inseridos pela API.

Passo 5: conferir no painel



SQL Editor: rode consultas e scripts (como o `Script_Livraria.sql` do projeto) direto no banco.

Passo 5: conferir no painel

Observação

O Supabase avisa quando uma tabela do schema `public` está com **RLS desativado** (*Row Level Security*). Isso importa quando o **navegador** acessa o banco pela API do Supabase (veremos na pasta Front-end). Quando só o Spring Boot acessa via JDBC com o usuário `postgres`, o RLS não interfere; mesmo assim, ativá-lo (`alter table livro enable row level security;`) evita que a API pública do Supabase exponha a tabela.

08

Aprofundando: relacionamentos e desempenho com JPA

Aprofundando: relacionamentos e desempenho com JPA

Um autor tem muitos livros

```java
@Entity
public class Autor {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;

    @OneToMany(mappedBy = "autor")
    @JsonIgnore                          // evita loop infinito no JSON
    private List<Livro> livros = new ArrayList<>();
}

@Entity
public class Livro {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String titulo;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "autor_id")       // chave estrangeira na tabela livro
    private Autor autor;
}
```

Aprofundando: relacionamentos e desempenho com JPA

Anotação	Lado	Tabela no banco
`@ManyToOne`	Muitos livros → um autor	Coluna `autor_id` em `livro`
`@OneToMany(mappedBy)`	Um autor → lista de livros	Nenhuma coluna nova (lado inverso)
`@OneToOne`	Um usuário → um perfil	Chave estrangeira única
`@ManyToMany`	Alunos ⇄ disciplinas	Tabela intermediária `aluno_disciplina`

JPQL x SQL nativo

JPQL (`@Query`)

- Usa nomes de **classes e atributos**
- `select l from Livro l where l.autor.nome = :n`
- Funciona em qualquer banco suportado

vs

SQL nativo (`nativeQuery = true`)

- Usa nomes de **tabelas e colunas**
- `select * from livro where autor_id = ?1`
- Acesso a recursos específicos do PostgreSQL

O problema N+1

Listar 50 livros e, para cada um, acessar `livro.getAutor().getNome()` pode disparar **51 consultas**: uma para os livros e uma por autor. É o problema N+1. Ligue `spring.jpa.show-sql=true` e você verá a repetição no console. A solução é buscar tudo de uma vez com `join fetch` ou `@EntityGraph`.

08

Aprofundando: relacionamentos e desempenho com JPA

Aula 06 · Back-end

O problema N+1

repository/LivroRepository.java

```java
@Query("select l from Livro l join fetch l.autor")
List<Livro> listarComAutor();            // 1 consulta só

@EntityGraph(attributePaths = "autor")
List<Livro> findByTituloContaining(String t);
```

Aprofundando: relacionamentos e desempenho com JPA

Tipo no PostgreSQL	Tipo em Java
`bigint` / `bigserial`	`Long`
`integer`	`Integer`
`varchar(n)` / `text`	`String`
`numeric(10,2)`	`BigDecimal` (dinheiro: nunca `double`)
`boolean`	`boolean` / `Boolean`
`timestamptz`	`OffsetDateTime` / `Instant`
`uuid`	`UUID`

Migrações com Flyway

Em produção, deixar o Hibernate alterar tabelas (`ddl-auto=update`) é arriscado. O **Flyway** aplica scripts SQL versionados na ordem certa, e cada script roda uma única vez por banco. É o caminho para quando o projeto deixa de ser de aula.

Flyway com Supabase

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-flyway</artifactId>
</dependency>
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-database-postgresql</artifactId>
</dependency>

-- src/main/resources/db/migration/V1__cria_livro.sql
create table livro (id bigserial primary key, titulo varchar(120) not null, ano int);

# application.properties
spring.jpa.hibernate.ddl-auto=validate
```

Aprofundando: relacionamentos e desempenho com JPA

Atenção

No plano gratuito do Supabase o *pooler* aceita poucas conexões simultâneas. Mantenha `spring.datasource.hikari.maximum-pool-size` baixo (5) e **não rode** várias instâncias da API apontando para o mesmo banco gratuito.

09

Quiz de revisão

Pergunta

Qual método de conexão do Supabase usar no Spring Boot e no Render?

A Direct connection

B Transaction pooler

C Session pooler

D Qualquer um

Resposta

Qual método de conexão do Supabase usar no Spring Boot e no Render?

A Direct connection

B Transaction pooler

✓ Session pooler

D Qualquer um

Por quê: A Direct connection é IPv6; o Session pooler funciona em IPv4 e com aplicações que ficam ligadas.

Quiz · questão 2 de 5

Pergunta

O usuário do Session pooler tem qual formato?

A postgres

B postgres.

C admin

D o e-mail da conta

Resposta

O usuário do Session pooler tem qual formato?

A postgres

✓ postgres.

C admin

D o e-mail da conta

Por quê: Sem o sufixo do projeto a autenticação falha.

Pergunta

O que `spring.jpa.hibernate.ddl-auto=update` faz?

A. Apaga o banco a cada execução

B. Cria tabelas e colunas que faltam

C. Só valida

D. Desliga o JPA

Resposta

O que `spring.jpa.hibernate.ddl-auto=update` faz?

A. Apaga o banco a cada execução

✓ **Cria tabelas e colunas que faltam**

C. Só valida

D. Desliga o JPA

Por quê: Ótimo para aula; em produção prefira validate + migrações.

Pergunta

Qual tipo Java usar para dinheiro?

A double

B float

C BigDecimal

D int

Resposta

Qual tipo Java usar para dinheiro?

A double

B float

✓ BigDecimal

D int

Por quê: double tem erros de arredondamento.

Pergunta

O que é o problema N+1?

A Ter N tabelas

B Uma consulta para a lista e mais uma para cada item relacionado

C Um erro de CORS

D Um limite do Supabase

Resposta

O que é o problema N+1?

A. Ter N tabelas

✓ Uma consulta para a lista e mais uma para cada item relacionado

C. Um erro de CORS

D. Um limite do Supabase

Por quê: Resolva com `join fetch` ou `@EntityGraph`.

Fechamento

Resumo da aula

- ✓ Supabase = PostgreSQL gerenciado na nuvem, com painel, Table Editor e SQL Editor.
- ✓ Use a *connection string* **JDBC** do **Session pooler** (IPv4); a senha vai em variável de ambiente.
- ✓ Dependências: `spring-boot-starter-data-jpa` + driver `postgresql`.
- ✓ `@Entity` mapeia a tabela; `JpaRepository` dá o CRUD; `ddl-auto=update` cria as tabelas.
- ✓ Confira os dados no Table Editor e rode SQL no SQL Editor.

Exercícios

1. Crie um projeto no Supabase e conecte o CRUD de `Livro` da aula anterior.

2. Rode o `Script_Livraria.sql` do projeto `BD_RestAPI_PostgreSQL` no SQL Editor e consuma os dados pela API.

3. Crie a entidade `Autor` e relacione com `Livro` usando `@ManyToOne`.

4. Troque `ddl-auto` para `validate` e explique o que acontece se você renomear um atributo.

Para ir além

- supabase.com/docs/guides/database/connecting-to-postgres
- Projeto `PROJETOS/SpringBoot/BD_RestAPI_PostgreSQL`

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 06?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web